
A Reproducible Comparison of Open-Vocabulary Object Detection and Visual Grounding Models

Zheng Renjie Ling Hongyu He Jiayang Deng Haotian

Abstract

Open-vocabulary object detection replaces a detector’s fixed label set with arbitrary text prompts, while visual grounding localizes an object described by a natural-language expression. We reproduce and compare three representative pretrained systems: OWL-ViT Base, Grounding DINO Tiny, and YOLO-World v2 Small. All models are evaluated without task-specific fine-tuning on the same seeded random subset of 500 COCO val2017 images using the 80 category names as prompts. Grounding DINO obtains the best COCO AP of 0.452, compared with 0.394 for YOLO-World and 0.270 for OWL-ViT. YOLO-World is substantially faster, reaching 62.0 pipeline frames per second on an RTX 4060 Laptop GPU, versus 11.6 for OWL-ViT and 1.5 for Grounding DINO. We additionally evaluate genuine referring-expression localization on a 100-expression RefCOCO validation subset. Grounding DINO, OWL-ViT, and YOLO-World reach top-1 Accuracy@IoU ≥ 0.5 of 59%, 56%, and 46%, respectively. Threshold and non-maximum suppression ablations show that aggressive filtering improves visual clarity but reduces recall and quantitative accuracy. The resulting codebase provides reproducible data preparation, evaluation, timing, and visualization scripts and exposes a clear accuracy–efficiency trade-off among the three model families.

1. Introduction

Conventional object detectors predict from a category vocabulary fixed during training. Adding a new concept generally requires collecting annotations and retraining or fine-tuning the model. Open-vocabulary object detection (OVD) instead conditions localization on text, enabling prompts such as “remote control,” “the red car,” or concepts not explicitly represented by a fixed output head. Visual grounding is closely related but more demanding: the input is often a full referring expression whose attributes and spatial relations must identify one instance among several candidates.

This project studies how readily available pretrained models behave under a common, reproducible protocol. We compare OWL-ViT (Minderer et al., 2022), Grounding DINO (Liu et al., 2024), and YOLO-World (Cheng et al., 2024). These systems span three useful design points: a simple vision-transformer transfer approach, a tightly fused language-conditioned transformer detector, and a real-time YOLO-style detector.

Our work makes four practical contributions:

1. a unified evaluation on the same fixed random 500-image COCO subset;
2. synchronized runtime and peak GPU-memory measurements;
3. a separate RefCOCO experiment that tests phrase-level grounding rather than only category-name detection; and
4. threshold and NMS ablations that quantify common deployment choices.

We do not claim a new detector. The goal is accurate reproduction, controlled comparison, and analysis of when each approach works or fails.

2. Related Work

Vision-language pretraining. CLIP learns transferable visual representations by matching images and natural language at scale (Radford et al., 2021). Its shared embedding space makes text an effective interface for downstream recognition, but image-level alignment does not by itself provide object boxes.

Open-vocabulary detection. OWL-ViT transfers a contrastively pretrained vision transformer to text-conditioned detection with minimal architectural changes (Minderer et al., 2022). It embeds each prompt and scores image tokens against the resulting text features. Grounding DINO builds on transformer detection ideas, including DETR (Carion et al., 2020), and introduces language-guided query selection plus cross-modal fusion (Liu et al.,

2024). YOLO-World targets real-time operation through a re-parameterizable vision-language path aggregation network and a prompt-then-detect workflow (Cheng et al., 2024).

Referring-expression comprehension. RefCOCO and related datasets pair COCO objects with expressions intended to distinguish a target from other objects in the same image (Yu et al., 2016). Unlike category detection, success may require color, position, attributes, or context. The conventional metric counts a prediction as correct when its intersection-over-union (IoU) with the target box exceeds 0.5.

3. Approach

3.1. Unified Text-Conditioned Pipeline

Given an image I and prompts $T = \{t_1, \dots, t_K\}$, each model produces candidate boxes, text labels, and confidence scores:

$$f_\theta(I, T) = \{(b_i, \hat{t}_i, s_i)\}_{i=1}^N. \quad (1)$$

We preserve each model’s official preprocessing and pre-trained weights. No training or fine-tuning is performed. Boxes are converted to COCO’s (x, y, w, h) format and evaluated with `pycocotools`.

OWL-ViT BASE

We use `google/owlvit-base-patch32`. For COCO, all 80 category names are passed as independent text queries. Predictions below 0.01 are removed and the top 100 are retained. Extra NMS is disabled in the main experiment because the model’s dense hypotheses are already ranked and our ablation showed that additional suppression reduced AP.

GROUNDING DINO TINY

We use `IDEA-Research/grounding-dino-tiny`. COCO class names are joined into a period-separated caption. Both box and text thresholds are 0.20, and the top 100 matched predictions are retained. The model is evaluated in FP32 because this configuration was stable on the Windows test environment.

YOLO-WORLD v2 SMALL

We use `yolov8s-worldv2.pt` through the Ultralytics interface (Jocher et al., 2023). The 80 COCO class embeddings are computed before inference. Images are resized according to the model’s 640-pixel input setting, with confidence threshold 0.001, internal NMS IoU 0.7, and at most 100 boxes.

3.2. Evaluation Metrics

For COCO detection, we report mean average precision over IoU thresholds 0.50:0.05:0.95 (AP), AP50, AP75, scale-specific AP, and average recall with at most 100 detections (AR100) (Lin et al., 2014). IoU is

$$\text{IoU}(B, \hat{B}) = \frac{|B \cap \hat{B}|}{|B \cup \hat{B}|}. \quad (2)$$

For RefCOCO, one expression is provided at a time and only the highest-scoring box is evaluated. We report

$$\text{Acc@}\tau = \frac{1}{M} \sum_{j=1}^M \mathbb{1}[\text{IoU}(B_j, \hat{B}_j) \geq \tau] \quad (3)$$

for $\tau \in \{0.5, 0.75\}$, together with mean IoU.

3.3. Runtime Measurement

Each model receives one warm-up image that is excluded from timing. CUDA is synchronized around preprocessing, inference, and postprocessing for OWL-ViT and Grounding DINO. YOLO-World’s internal stage timings are accumulated. File download and disk image loading are excluded. We report pipeline FPS and PyTorch’s peak allocated GPU memory. These numbers characterize our specific hardware and software configuration rather than universal model speed.

4. Experimental Setup

Hardware and software. Experiments run on an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM. The environment uses Python 3.10, PyTorch 2.11.0 with CUDA 12.8, Torchvision 0.26.0, Transformers 5.5.3, Ultralytics 8.4.70, and `pycocotools` 2.0.11.

COCO subset. We sample 500 images without replacement from COCO val2017 using Python’s fixed random seed 308. The sorted selected image IDs are stored with every metrics file, so all models see exactly the same images. The 80 COCO categories serve as text prompts. This is more representative than selecting the first 100 image IDs, but it remains a subset rather than a full COCO validation run.

RefCOCO subset. We use the first 100 region rows exposed by the `lmms-lab/RefCOCO` validation mirror, with original COCO train2014 images and target boxes. Each row contains two to four human expressions. We use the first expression per region to avoid repeatedly weighting the same target. This experiment is a small reproducible diagnostic and should not be compared directly with results on the complete official RefCOCO validation split.

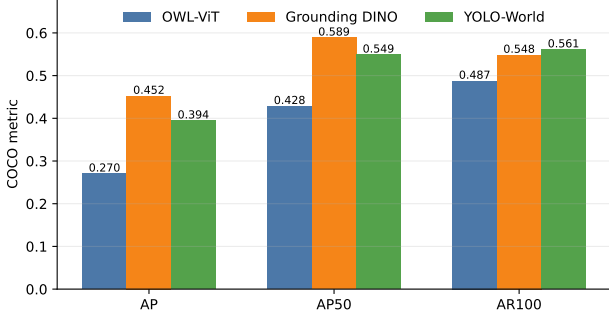


Figure 1: COCO detection accuracy on the shared 500-image subset.

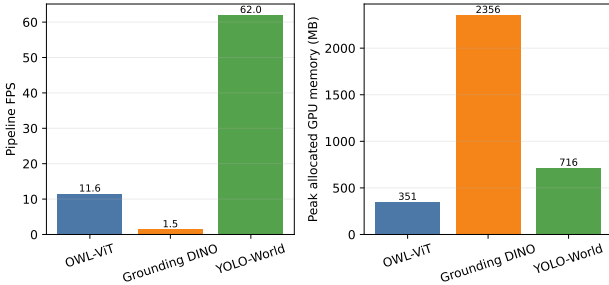


Figure 2: Pipeline throughput and peak allocated GPU memory.

5. Experimental Results

5.1. Open-Vocabulary Detection on COCO

Table 1 and Figure 1 show a clear ranking in AP. Grounding DINO exceeds YOLO-World by 5.8 AP points and OWL-ViT by 18.2 points. Its advantage is largest for small objects: AP_S is 0.325 versus 0.234 and 0.134. This supports the qualitative observation that small or cluttered instances are a major weakness of OWL-ViT. YOLO-World has the highest AR100 and AP_L , indicating strong proposal coverage and localization for larger objects, although its lower overall AP suggests more ranking or classification errors than Grounding DINO.

5.2. Accuracy–Efficiency Trade-off

YOLO-World is the practical speed winner (Table 2 and Figure 2). It is 5.4 times faster than OWL-ViT and over 42 times faster than Grounding DINO in pipeline FPS. Grounding DINO uses the most memory and is the slowest, but provides the highest AP. OWL-ViT uses the least allocated GPU memory, although its accuracy is lowest. Thus no model dominates: Grounding DINO is preferable when accuracy is primary, YOLO-World when latency is constrained, and OWL-ViT when a simple low-memory transformer baseline is desired.

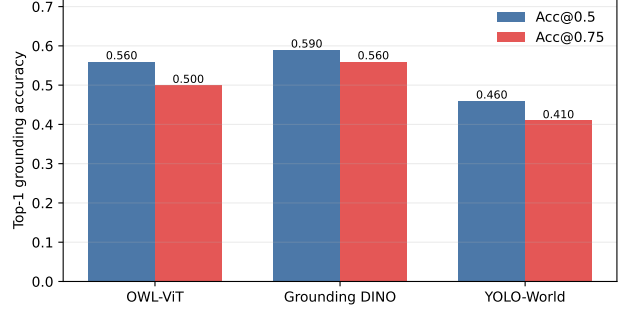


Figure 3: RefCOCO top-1 Accuracy@IoU thresholds.

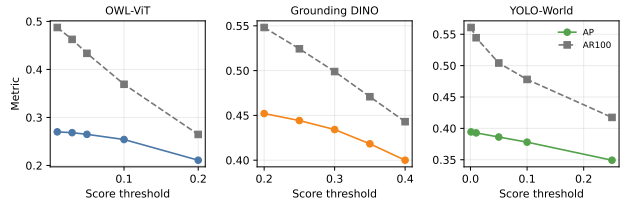


Figure 4: Within-model score-threshold sensitivity on the COCO subset.

5.3. Referring-Expression Grounding

Grounding DINO remains strongest on phrase-level localization (Table 3, Figure 3), consistent with its explicit cross-modal fusion and grounded pretraining. OWL-ViT is only three points behind at IoU 0.5 and remains a competitive zero-shot grounding baseline. YOLO-World is faster but loses 13 points relative to Grounding DINO, suggesting that longer relational descriptions are harder for its efficiency-oriented text-conditioned detector.

5.4. Threshold Sensitivity and NMS

The three model APIs expose differently calibrated scores, so a numerical threshold of 0.2 does not have the same meaning across models. We therefore vary each threshold only within a model, using saved predictions. As shown in Figure 4, AP and AR decrease monotonically as detections are removed. For OWL-ViT, increasing the threshold from 0.01 to 0.20 reduces AP from 0.270 to 0.211 and AR100 from 0.487 to 0.265. Grounding DINO drops from 0.452 to 0.400 AP between 0.20 and 0.40. YOLO-World drops from 0.394 to 0.349 between 0.001 and 0.25. Higher thresholds can still be useful for uncluttered visual demonstrations, but they are harmful when recall matters.

We also evaluated class-wise NMS for OWL-ViT on the earlier 100-image subset. At NMS IoU 0.5, AP decreased from 0.336 to 0.323 and AR100 from 0.520 to 0.475. NMS visually removes duplicate boxes, but overlapping hypotheses can be valid in crowded scenes. We therefore disable addi-

Table 1: Zero-shot detection on the fixed random 500-image COCO val2017 subset. Best values are bold.

Model	AP	AP50	AP75	AP _S	AP _M	AP _L	AR100
OWL-ViT Base	0.270	0.428	0.287	0.134	0.303	0.454	0.487
Grounding DINO Tiny	0.452	0.589	0.498	0.325	0.497	0.549	0.548
YOLO-World v2 Small	0.394	0.549	0.427	0.234	0.452	0.556	0.561

Table 2: Measured efficiency on the same GPU. Pipeline time excludes disk I/O.

Model	ms/image	FPS	Peak MB
OWL-ViT Base	86.6	11.6	351
Grounding DINO Tiny	680.5	1.5	2356
YOLO-World v2 Small	16.1	62.0	716

Table 3: Top-1 localization on 100 RefCOCO validation expressions.

Model	Acc@0.5	Acc@0.75	Mean IoU
OWL-ViT Base	0.56	0.50	0.513
Grounding DINO Tiny	0.59	0.56	0.591
YOLO-World v2 Small	0.46	0.41	0.468

tional NMS for the main OWL-ViT quantitative comparison and use it only for selected visualizations.

5.5. Qualitative Analysis

Figure 5 compares representative OWL-ViT and Grounding DINO outputs. Both models handle common category nouns and multi-word prompts. Grounding DINO generally produces tighter boxes and fewer missed small objects. Common failure modes include duplicate low-confidence boxes, confusion between nearby objects, failure to resolve spatial relations, and missed small accessories. Prompt wording also matters: a broad noun can activate several instances, while a detailed expression can either disambiguate the target or reduce confidence if its phrasing differs from pretraining language.

6. Limitations

First, 500 COCO images reduce computation but produce less stable estimates than the full 5,000-image validation set. Although fixed random sampling avoids the obvious ordering bias of the original first-100 protocol, subset variance remains. Second, COCO’s 80 names are likely represented in the models’ large-scale pretraining data. The experiment therefore measures zero-shot transfer without COCO-specific fine-tuning, not guaranteed generalization to truly unseen concepts.

Third, the RefCOCO experiment uses only 100 mirrored

validation rows and one expression per target. A full evaluation should use every official split and all expressions. Fourth, score calibration differs across architectures; threshold sensitivity is meaningful within a model but cannot fully equalize precision–recall operating points across models. Finally, runtime depends on software versions, input resolution, precision, and hardware. In particular, Grounding DINO was kept in FP32 for stability, while OWL-ViT used FP16 on GPU.

7. Conclusion

We reproduced three open-vocabulary detection systems and evaluated them under a common protocol. Grounding DINO Tiny provides the best detection and grounding accuracy, reaching 0.452 COCO AP and 59% RefCOCO Acc@0.5. YOLO-World v2 Small offers the strongest efficiency–accuracy compromise, reaching 0.394 AP at 62.0 pipeline FPS. OWL-ViT is compact and simple but trails on COCO, especially for small objects. The experiments also show that postprocessing choices are not cosmetic: high confidence thresholds and extra NMS can make figures cleaner while reducing AP and recall.

Future work should evaluate the full COCO and RefCOCO splits, add LVIS rare categories or self-collected non-COCO objects to better test unseen concepts, and compare prompt paraphrases systematically. Calibration or per-model precision–recall operating-point selection would also improve deployment comparisons.

Team Contributions

All four members contributed equally (25% each).

References

- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., and Zagoruyko, S. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pp. 213–229, 2020.
- Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X., and Shan, Y. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16901–16911, 2024.

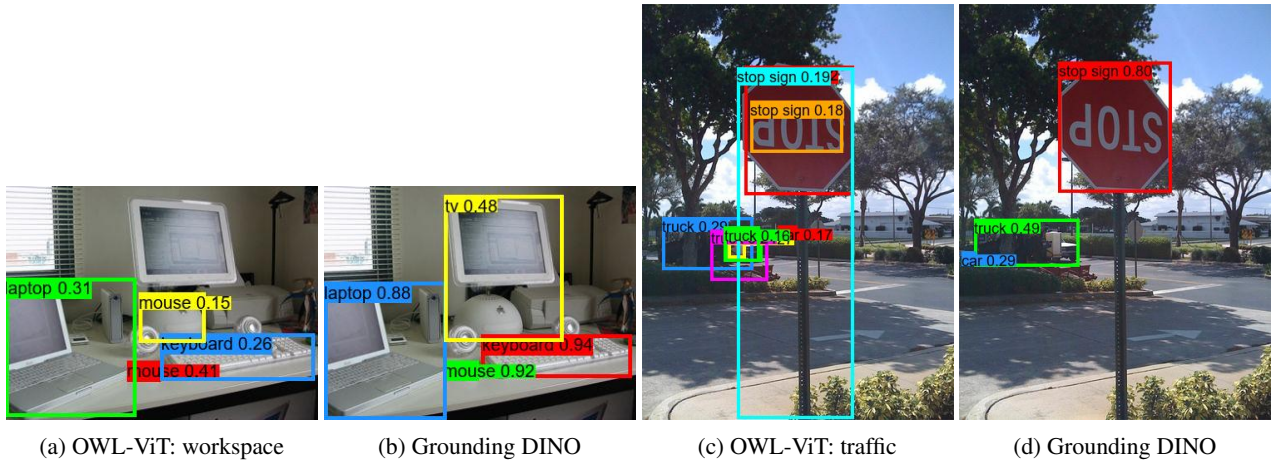


Figure 5: Qualitative open-vocabulary detections. Display thresholds and NMS are chosen for readability and differ from the quantitative protocol.

Jocher, G., Chaurasia, A., and Qiu, J. Ultralytics YOLO. *GitHub repository*, 2023. URL <https://github.com/ultralytics/ultralytics>.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, pp. 740–755, 2014.

Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Jiang, Q., Li, C., Yang, J., Su, H., Zhu, J., and Zhang, L. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, pp. 38–55, 2024.

Minderer, M., Gritsenko, A., Stone, A., Neumann, M., Weissenborn, D., Dosovitskiy, A., Mahendran, A., Arnab, A., Dehghani, M., Shen, Z., Wang, X., Zhai, X., Kipf, T., and Houlsby, N. Simple open-vocabulary object detection with vision transformers. In *European Conference on Computer Vision*, pp. 728–755, 2022.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pp. 8748–8763, 2021.

Yu, L., Poirson, P., Yang, S., Berg, A. C., and Berg, T. L. Modeling context in referring expressions. In *European Conference on Computer Vision*, pp. 69–85, 2016.